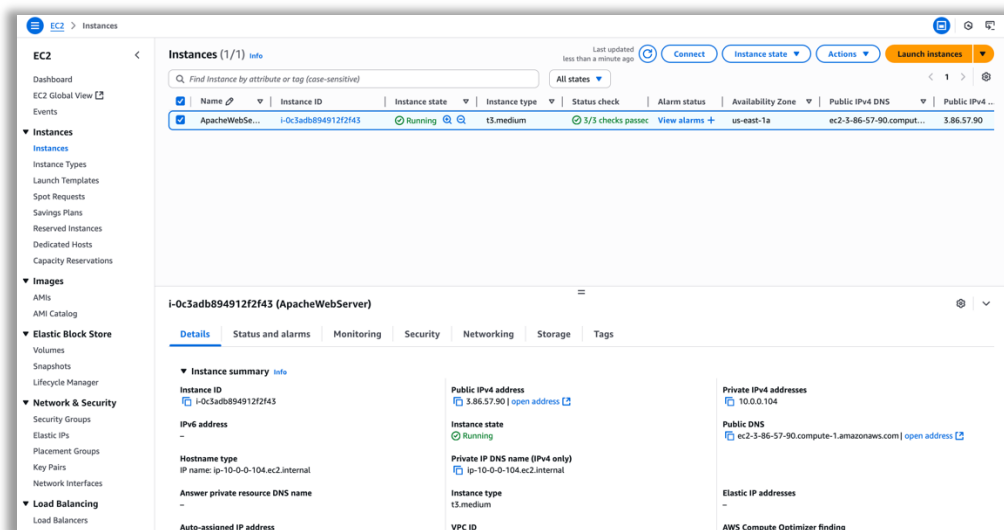


Creating a Custom AMI and Deploying an Auto Scaling Group behind an Application Load Balancer – Christopher Samugana

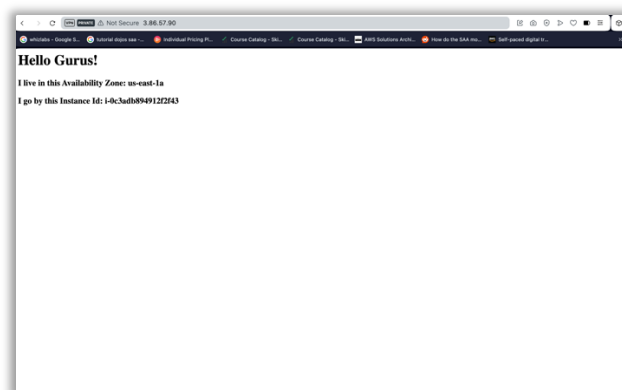
Introduction

In this lab, I built a functional web server on Amazon EC2, created a custom Amazon Machine Image (AMI) from the configured instance, and deployed an Auto Scaling Group (ASG) using the AMI. The ASG was configured behind an Application Load Balancer (ALB) to distribute traffic and support scalability.

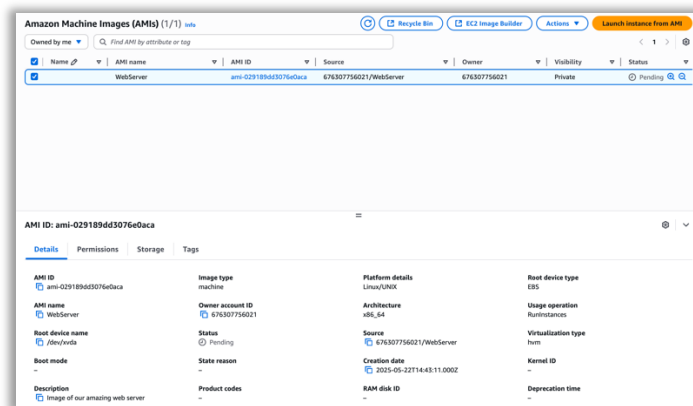
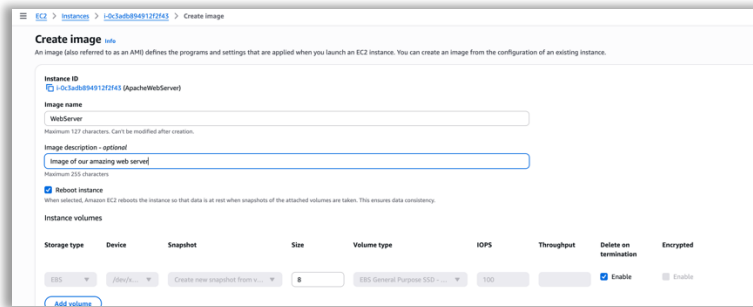
1. Created an AMI of the Amazon EC2 Instance



I began by logging into the AWS Management Console and confirmed the us-east-1 region was selected.

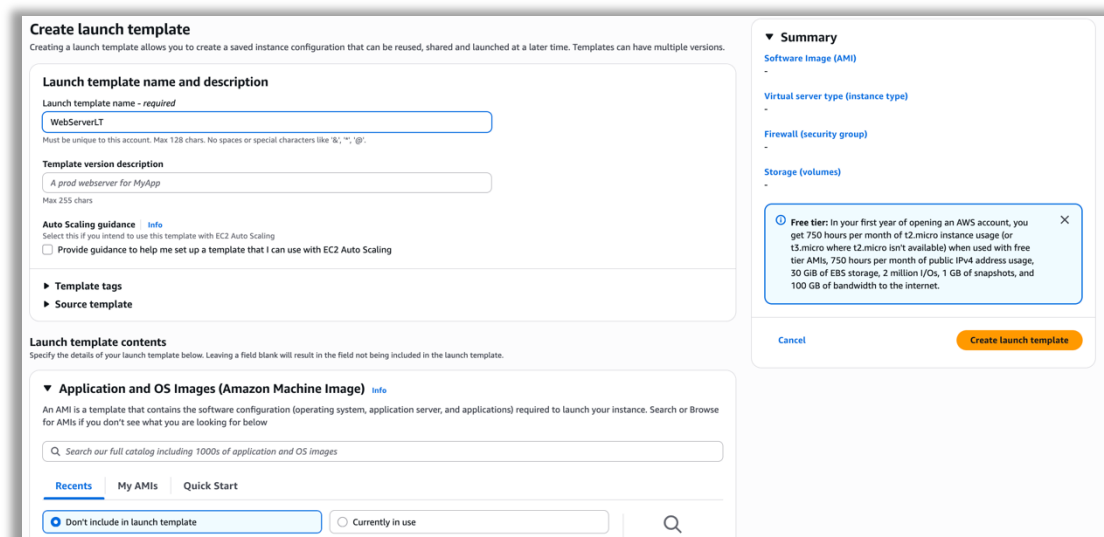


From the EC2 dashboard, I located the ApacheWebServer instance and verified that the web page was accessible by navigating to the HTTP version of the instance's public IPv4 address. If the page didn't load, I restarted the instance and confirmed accessibility again.



Once the page was working, I selected the instance, went to Actions > Images and templates > Create image, and named the AMI WebServer. I kept all other settings as defaults and created the image. I then navigated to the AMIs section under the Images menu and waited for the new WebServer AMI to complete its creation. This process took several minutes.

2. Created Launch Template



Next, I created a Launch Template (LT) that would utilize the new AMI. From the EC2 console, I went to Launch Templates, then clicked Create launch template and named it WebServerLT.

Step 1: Choose launch template or configuration

Choose launch template or configuration

Specify a launch template that contains settings common to all EC2 instances that are launched by this Auto Scaling group. If you currently use launch configurations, you might consider migrating to launch templates.

Name
Auto Scaling group name
Enter a name to identify the group.
WebServer-asg
Must be unique to this account in the current Region and no more than 255 characters.

Launch template
Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.
WebServerLT
Create a launch template

Version
Default (1)
Create a launch template version

Description
-

Launch template
WebServerLT
lt-05e54497c0b17d464

Instance type
t3.micro

AMI ID
ami-029189dd3076e6aca

Security groups
-

Security group IDs
sg-09ce400bfa08248

Key pair name
-

Additional details

Storage (volumes)
-

Date created
Thu May 22 2025 15:51:06 GMT+0100 (British Summer Time)

Cancel Next

Under Application and OS Images, I selected the My AMIs tab and chose the WebServer AMI. I then set the Instance type to t3.micro.

Step 3 - optional: Integrate with other services

Integrate with other services - optional

Use a load balancer to distribute network traffic across multiple servers. Enable service-to-service communications with VPC Lattice. Shift resources away from impaired Availability Zones with zonal shift. You can also customize health check replacements and monitoring.

Load balancing
Use the options below to attach your Auto Scaling group to an existing load balancer, or to a new load balancer that you define.
☐ No load balancer
Traffic to your Auto Scaling group will not be fronted by a load balancer.
☐ Attach to an existing load balancer
Choose from your existing load balancers.
☒ Attach to a new load balancer
Quickly create a basic load balancer to attach to your Auto Scaling group.

Attach to a new load balancer
Define a new load balancer to create for attachment to this Auto Scaling group.

Load balancer type
Choose from the load balancer types offered below. Type selection cannot be changed after the load balancer is created. If you need a different type of load balancer than those offered here, visit the [Load Balancing console](#).
☒ Application Load Balancer
HTTP, HTTPS
☐ Network Load Balancer
TCP, UDP, TLS

Load balancer name
Name cannot be changed after the load balancer is created.
WebServer-asg-1

Load balancer scheme
Scheme cannot be changed after the load balancer is created.
☐ Internal
☒ Internet-facing

Network mapping
Your new load balancer will be created using the same VPC and Availability Zone selections as your Auto Scaling group. You can select different subnets and add subnets from additional Availability Zones.

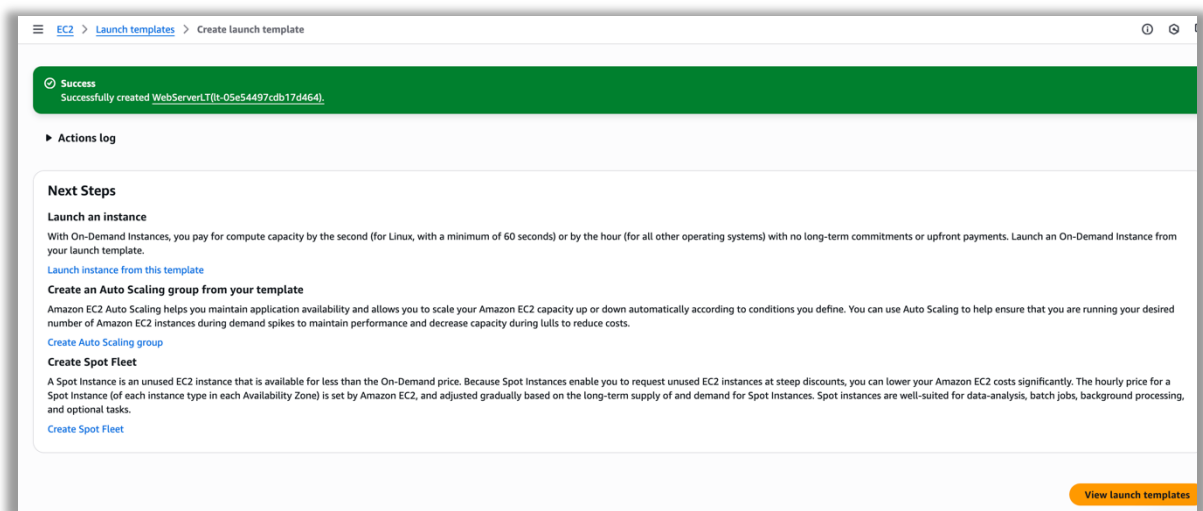
VPC
vpc-0553cbb3a6c64c1
Your Custom VPC

Availability Zones and subnets
You must select a single subnet for each Availability Zone enabled. Only public subnets are available for selection to support DNS resolution.
☒ us-east-1b
subnet-083422a82d825e9e0
☒ us-east-1c
subnet-0f877af651c7c7f79
☒ us-east-1a
subnet-0598d374f2dcce683

Listeners and routing
If you require secure listeners, or multiple listeners, you can configure them from the [Load Balancing console](#) after your load balancer is created.

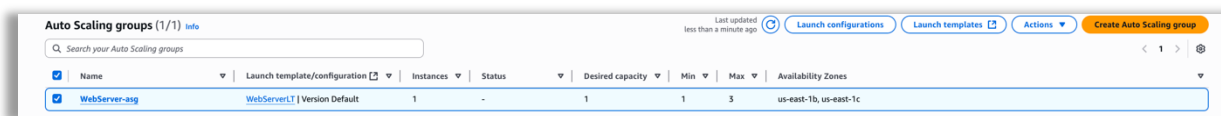
For Key pair (login) and Subnet, I opted not to include these in the launch template. I created a new Security Group named AllowVpcHTTP and selected the custom VPC with a CIDR block of 10.0.0.0/16. I added an inbound rule allowing HTTP (TCP port 80) traffic from within the VPC (10.0.0.0/16).

In the Advanced details, I selected the IAM role OurSystemsManagerEC2Profile and enabled Detailed CloudWatch monitoring. I also added user data to install Apache, stress tools, and generate a dynamic HTML page that displayed the instance ID and availability zone.



After reviewing all settings, I created the launch template.

3. Created Auto Scaling Group and Load Balancer



I then used the newly created launch template to configure an Auto Scaling Group. From the Launch Templates section, I selected WebServerLT, clicked on Actions, and chose Create Auto Scaling group. I named it WebServer-asg.

For the network settings, I selected the custom VPC and added all private subnets for availability.

For Load balancing, I chose to attach a new Application Load Balancer, set it to Internet-facing, and made sure the subnets were all public. I also created a new Target Group, left the default name, and disabled Elastic Load Balancer health checks. I set the health check grace period to 60 seconds.

For Group size, I set:

- Desired capacity: 1
- Min: 1
- Max: 3

I created a Target Tracking Scaling Policy, set the metric type to Average CPU Utilization, and defined the target value as 50% with a 60-second warmup period.

I skipped the rest of the menus and finalized the Auto Scaling Group creation.

Once complete, I verified that a new instance was launched under the Instance management tab. I also ensured that the ALB was in the active state.

4. Updated ALB Security Group

Security groups (1)

A security group is a set of firewall rules that control the traffic to your load balancer.

Security Group ID 🔗	Name	Description
sg-03b9801c9ed39c3c2	default	default VPC security group

After confirming the ALB was created, I noticed it used the same Security Group as the EC2 instances, which did not allow external internet traffic.

Create security group [Info](#)

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details

Security group name [Info](#)

Name cannot be edited after creation.

Description [Info](#)

VPC [Info](#)

To fix this, I created a new Security Group named OurALBSecurityGroup, added an inbound rule allowing HTTP traffic from Anywhere (0.0.0.0/0), and associated it with the custom VPC.

Load balancer details: WebServer-asg-1

Security groups

A security group is a set of firewall rules that control the traffic to your load balancer. Select an existing security group, or you can [create a new security group](#).

Security groups
Select up to 5 security groups

OurALBSecurityGroup
sg-02044b95c628d1f1e

VPC: vpc-0353bcb3ae6c64cd

default
sg-03b9801c9ed39c3c2

VPC: vpc-0353bcb3ae6c64cd

[Cancel](#) [Save changes](#)

Edit security groups

Load balancer details: WebServer-asg-1

Security groups

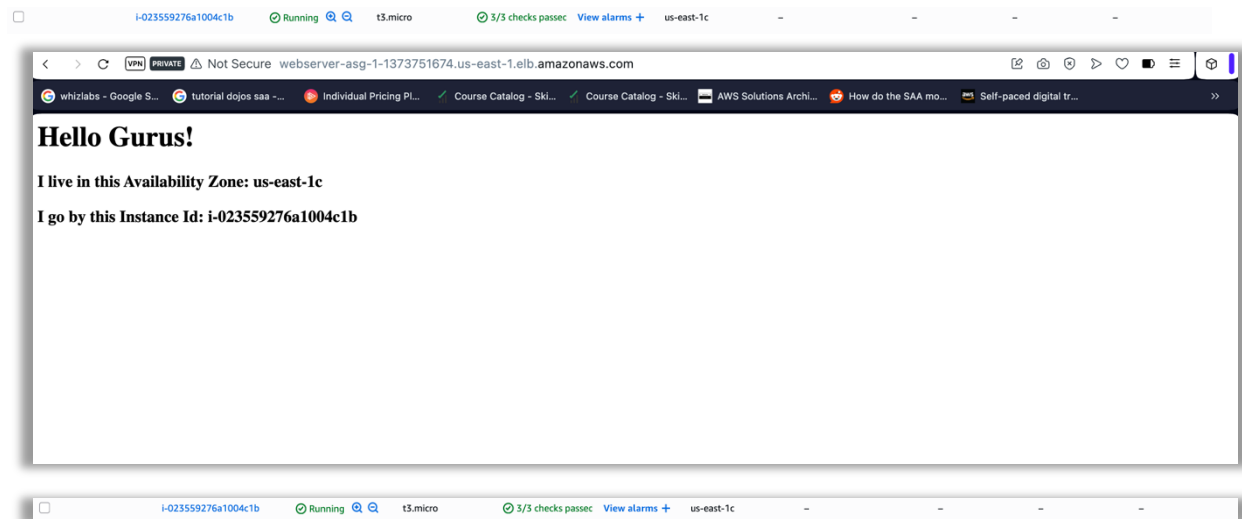
A security group is a set of firewall rules that control the traffic to your load balancer. Select an existing security group, or you can [create a new security group](#).

Security groups
Select up to 5 security groups

OurALBSecurityGroup
sg-02044b95c628d1f1e

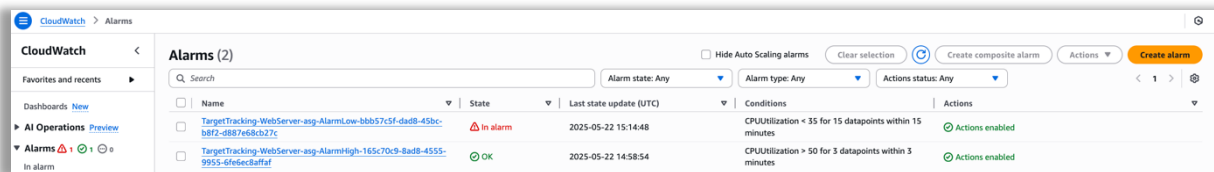
VPC: vpc-0353bcb3ae6c64cd

I returned to the ALB's Security tab, removed the default Security Group, and attached the newly created one.

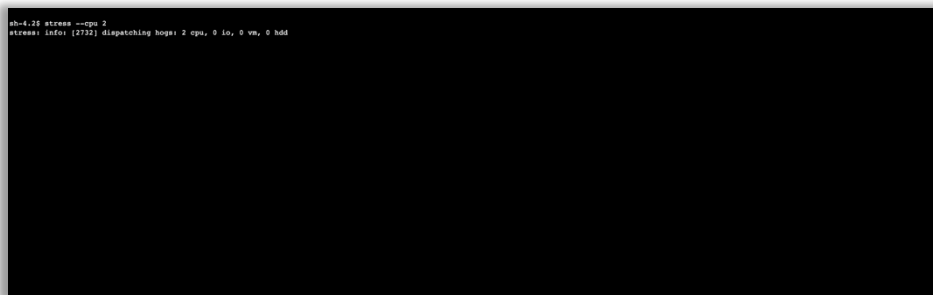
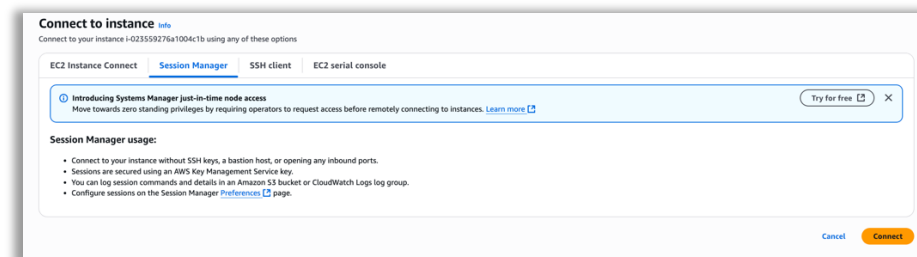


I then copied the DNS name of the ALB, opened it in a browser using HTTP, and verified that the application loaded successfully.

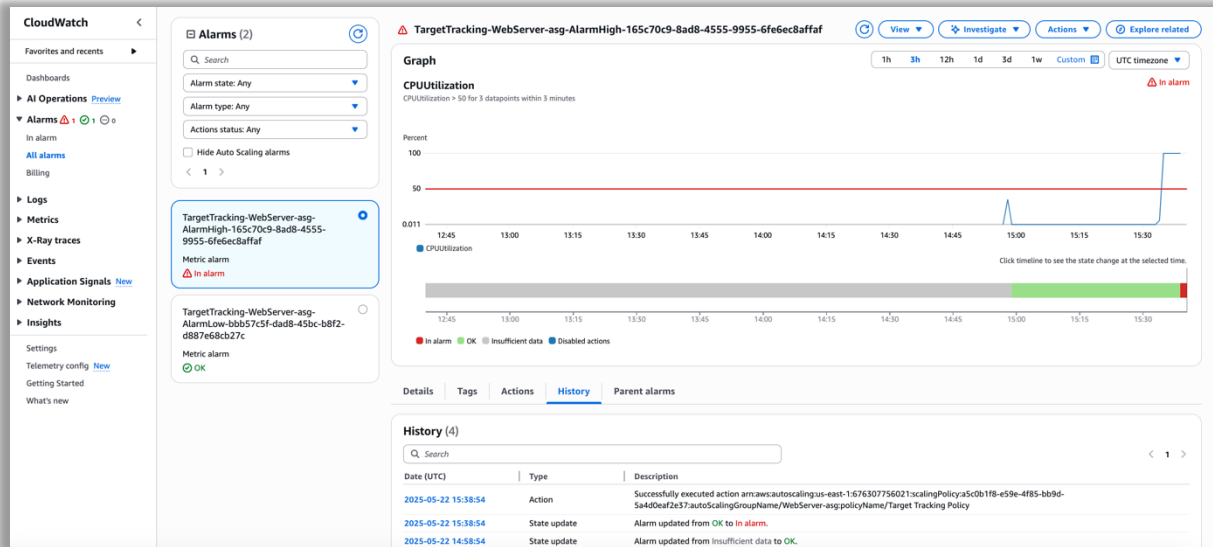
5. Tested Auto Scaling



To test the scaling functionality, I navigated to Amazon CloudWatch > Alarms > All alarms and confirmed the two alarms created by the policy.



I then connected to the running instance in the ASG using Session Manager (enabled via the NAT Gateway in the private subnet), and ran the pictured command to simulate CPU



Back in CloudWatch, I observed the alarm eventually entered the In Alarm state. I returned to the Auto Scaling Group dashboard, clicked on Activity, and saw that a new instance was being added.

The screenshot displays the AWS Auto Scaling groups console. The table shows one group, 'WebServer-asg', in a 'Running' state. The table has columns for Name, Launch template/configuration, Instances, Status, Desired capacity, Min, Max, and Availability Zones.

Name	Launch template/configuration	Instances	Status	Desired capacity	Min	Max	Availability Zones
WebServer-asg	WebServerLT Version Default	3	Running	3	1	3	us-east-1b, us-east-1c

I verified in the ALB that an additional instance was registered. After refreshing the ALB DNS, the traffic was served by different EC2 instances, confirming load balancing. (Optional) Once I stopped the stress process, the CPU metrics decreased, and after some time, the ASG began scaling down as expected.

WebServer-asg

WebServer-asg Capacity overview

arn:aws:autoscaling:us-east-1:676307756021:autoScalingGroup:3619a9ff-dfde-4b0c-94d6-ccdea335f89e:autoScalingGroupName/WebServer-asg

Desired capacity

3

Scaling limits (Min - Max)

1 - 3

Desired capacity type

Units (number of Instances)

Status

-

Date created

Thu May 22 2025 15:57:00 GMT+0100 (British Summer Time)

Details

Integrations - new

Automatic scaling

Instance management

Instance refresh

Activity

Monitoring

Activity notifications (0)

Filter notifications

Send to

On instance action

No notifications are currently specified

Create notification

Activity history (4)

Filter activity history

Status	Description	Cause	Start time	End time
Successful	Launching a new EC2 instance: i-0f5d729d34f19d623	At 2025-05-22T15:40:54Z a monitor alarm TargetTracking-WebServer-asg-AlarmHigh-165c70c9-8ad8-4555-9955-6fe6ec8affaf in state ALARM triggered policy Target Tracking Policy changing the desired capacity from 2 to 3. At 2025-05-22T15:40:59Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 2 to 3.	2025 May 22, 04:41:01 PM +01:00	2025 May 22, 04:42:10 PM +01:00
Successful	Launching a new EC2 instance: i-028641c645baf1140	At 2025-05-22T15:38:54Z a monitor alarm TargetTracking-WebServer-asg-AlarmHigh-165c70c9-8ad8-4555-9955-6fe6ec8affaf in state ALARM triggered policy Target Tracking Policy changing the desired capacity from 1 to 2. At 2025-05-22T15:39:08Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 1 to 2.	2025 May 22, 04:39:10 PM +01:00	2025 May 22, 04:40:15 PM +01:00
Successful	Updating load balancers/target groups: Successful. Status Reason: Added: arn:aws:elasticloadbalancing:us-east-1:676307756021:targetgroup/WebServer-TG/cfe9b4fa036d90bb (Target Group).		2025 May 22, 03:57:10 PM +01:00	2025 May 22, 03:57:10 PM +01:00
Successful	Launching a new EC2 instance: i-023559276a1004c1b	At 2025-05-22T14:57:00Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 1. At 2025-05-22T14:57:00Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 1.	2025 May 22, 03:57:02 PM +01:00	2025 May 22, 03:57:07 PM +01:00

WebServer-TG

Details

arn:aws:elasticloadbalancing:us-east-1:676307756021:targetgroup/WebServer-TG/cfe9b4fa036d90bb

Target type

Instance

IP address type

IPv4

Protocol : Port

HTTP: 80

Load balancer

WebServer-asg-1

Protocol version

HTTP1

VPC

vpc-0353bcbb3ae6c64cd

3

Total targets

3

Healthy

0 Anomalous

0

Unhealthy

0

Unused

0

Initial

0

Draining

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Conclusion

This lab successfully demonstrated how to create a custom AMI from an EC2 instance, deploy it via a Launch Template into an Auto Scaling Group, and connect it behind an Application Load Balancer for high availability and scalability. The Auto Scaling policies worked as intended, automatically adjusting instance count based on CPU utilization thresholds.